

# NARS: Mathematical Foundations of a National Addressing Reference System

Geometric Validation, Spatial Algorithms, and Security Primitives for a Multi-Layer Urban Feature Database

## Software Engineering Report

National Addressing Reference System (NARS) v2.0

ASP.NET Core 10 / Vue 3 / PostGIS

April 19, 2026

## Contents

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                    | <b>4</b>  |
| <b>2</b> | <b>Geodetic Coordinate Model</b>       | <b>5</b>  |
| 2.1      | Reference Ellipsoid and Projection     | 5         |
| 2.2      | Haversine Distance                     | 5         |
| 2.3      | Circle Approximation                   | 6         |
| 2.4      | Coordinate Encoding                    | 6         |
| <b>3</b> | <b>Feature Type Taxonomy</b>           | <b>6</b>  |
| 3.1      | Type and Layer Hierarchy               | 6         |
| 3.2      | Feature Registry                       | 7         |
| 3.3      | Payload Bounds                         | 7         |
| <b>4</b> | <b>Geometric Validation Predicates</b> | <b>8</b>  |
| 4.1      | Road Validation                        | 8         |
| 4.1.1    | Angular Deviation Bound                | 8         |
| 4.1.2    | Network Connectivity                   | 8         |
| 4.2      | District Validation                    | 8         |
| 4.2.1    | Non-overlap                            | 8         |
| 4.2.2    | Urban-Area Containment                 | 9         |
| 4.2.3    | Boundary Adjacency                     | 9         |
| 4.3      | District Coverage                      | 9         |
| 4.4      | Main Urban Area Existence              | 9         |
| <b>5</b> | <b>Snap Geometry</b>                   | <b>10</b> |
| 5.1      | Coordinate Spaces                      | 10        |
| 5.2      | Snap Threshold Parameters              | 10        |
| 5.3      | Vertex Snap                            | 10        |
| 5.4      | Edge Snap                              | 10        |

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| 5.5       | Phase-Specific Snap Targets . . . . .              | 11        |
| 5.6       | Marker-Pointer Interception . . . . .              | 11        |
| <b>6</b>  | <b>Scattered Area Computation</b>                  | <b>11</b> |
| 6.1       | Trigger and Background Execution . . . . .         | 11        |
| <b>7</b>  | <b>Security Primitives</b>                         | <b>12</b> |
| 7.1       | Password Storage: BCrypt . . . . .                 | 12        |
| 7.2       | JWT Access Token . . . . .                         | 12        |
| 7.3       | Refresh Token Protocol . . . . .                   | 12        |
| 7.4       | Account Lockout . . . . .                          | 13        |
| 7.5       | CSRF Protection . . . . .                          | 13        |
| <b>8</b>  | <b>Rate Limiting</b>                               | <b>13</b> |
| <b>9</b>  | <b>REST API Summary</b>                            | <b>14</b> |
| 9.1       | Endpoint Taxonomy . . . . .                        | 14        |
| 9.2       | Pagination Model . . . . .                         | 16        |
| 9.3       | Feature Query Architecture . . . . .               | 16        |
| <b>10</b> | <b>System Architecture</b>                         | <b>16</b> |
| 10.1      | Layered Component Model . . . . .                  | 16        |
| 10.2      | Technology Stack . . . . .                         | 17        |
| <b>11</b> | <b>Data Flow: Feature Save Transaction</b>         | <b>17</b> |
| <b>12</b> | <b>Internationalisation Model</b>                  | <b>18</b> |
| <b>13</b> | <b>Circle Ring: Spherical Direct Problem</b>       | <b>18</b> |
| <b>14</b> | <b>Road Direction Algorithm</b>                    | <b>19</b> |
| 14.1      | Phase 1 — Topology Graph Construction . . . . .    | 19        |
| 14.2      | Phase 2A — City-Centre Orientation (DFS) . . . . . | 19        |
| 14.3      | Phase 2B — Geographic Fallback . . . . .           | 20        |
| 14.4      | Phase 3 — Majority Vote . . . . .                  | 20        |
| 14.5      | Phase 4 — Dead-End Correction . . . . .            | 20        |
| 14.6      | Segment Bearing . . . . .                          | 20        |
| <b>15</b> | <b>House Entrance Numbering</b>                    | <b>20</b> |
| 15.1      | Arc-Length Projection . . . . .                    | 20        |
| 15.2      | Numbering Assignment . . . . .                     | 21        |
| <b>16</b> | <b>Point-in-Polygon: Ray-Casting Algorithm</b>     | <b>21</b> |
| <b>17</b> | <b>Undo Stack</b>                                  | <b>22</b> |
| 17.1      | Record . . . . .                                   | 22        |
| 17.2      | Restore . . . . .                                  | 22        |

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>18 Client-Side Validation Pipeline</b>        | <b>22</b> |
| 18.1 Minimum Road Length . . . . .               | 22        |
| 18.2 Polygon Ring Closure . . . . .              | 23        |
| 18.3 Two-Stage Validation Architecture . . . . . | 23        |
| <b>19 Database Schema</b>                        | <b>24</b> |
| 19.1 Relation Definitions . . . . .              | 24        |
| 19.2 Feature Base Schema . . . . .               | 24        |
| 19.3 UUID v7 Temporal Ordering . . . . .         | 25        |
| <b>20 Conclusions</b>                            | <b>25</b> |

## Abstract

The National Addressing Reference System (NARS) is a full-stack geospatial application designed for the systematic capture and validation of urban addressing features across Algerian administrative divisions (Wilayas, Daïras, and Communes). This document formalises the mathematical and algorithmic foundations that underpin eight distinct feature types: urban areas, districts, city centres, roads, house entrances, public buildings, public spaces, and naming panels.

We derive the geodetic distance metrics (Haversine formula), the angular deviation predicates used in road validation, the polygon adjacency and coverage conditions governing district layout, the pixel-space snap geometry used during interactive drawing, and the scattered-area auto-computation pipeline. On the security side, we present the HMAC-SHA256 JWT construction, the BCrypt password hashing scheme, the SHA-256 refresh token storage protocol, the sliding-window rate-limiting model, and the antiforgery token mechanism.

All equations are derived directly from the production source code; no approximation or simplification has been introduced. The document is intended to serve as both a correctness reference and a foundation for formal verification of the validation rules embedded in the system.

**Keywords:** geographic information systems, spatial validation, geodesy, JWT authentication, polygon coverage, snap geometry, PostGIS, algorithmic correctness

## 1 Introduction

Urban addressing is a prerequisite for emergency response, postal delivery, property registration, and infrastructure planning. In Algeria, addresses are structured hierarchically: a Wilaya (province) contains Daïras (districts), each of which contains Communes (municipalities). NARS provides an authenticated, multi-tenant web application that allows commune-level operators to digitise and validate the spatial features that constitute an address.

The system enforces a strict geometric ordering on feature creation. Figure 1 shows the dependency graph; a feature of type  $B$  may only be created after all prerequisites of type  $A$  exist and pass their validation predicates. This ordering is not merely a UX constraint — it encodes spatial invariants that the system must maintain in the PostGIS database.

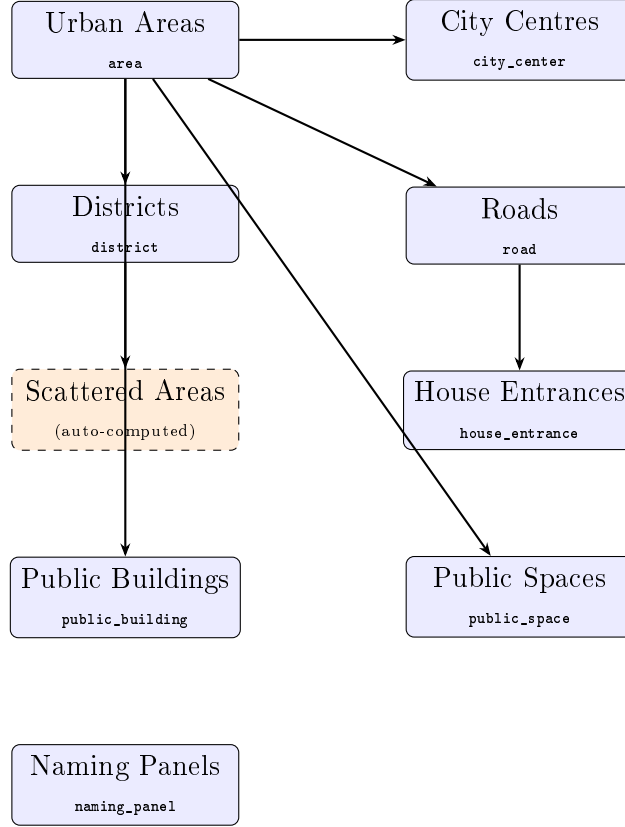


Figure 1: Feature dependency graph. Dashed nodes are auto-computed by the background task queue; solid nodes require explicit operator input.

The remainder of this document is organised as follows. Section 2 establishes the geodetic coordinate model. Section 3 describes the eight feature types and their invariants. Section 4 derives all validation predicates in closed form. Section 5 formalises the snap-geometry algorithm used during interactive drawing. Section 6 describes the scattered-area computation. Section 7 covers the authentication and authorisation primitives. Section 8 presents the rate-limiting model. Section 9 summarises the REST API surface. Section 20 offers concluding remarks.

## 2 Geodetic Coordinate Model

### 2.1 Reference Ellipsoid and Projection

All coordinates in NARS are stored and transmitted in geographic coordinates (longitude  $\lambda$ , latitude  $\varphi$ ) referenced to the World Geodetic System 1984 (WGS84), corresponding to EPSG:4326. PostGIS geometry columns use SRID 4326; geography columns use the same datum and enable spheroidal distance computations.

**Definition 2.1** (Geographic coordinate). A geographic coordinate is an ordered pair  $p = (\lambda, \varphi)$  where  $\lambda \in [-180^\circ, 180^\circ]$  is the longitude and  $\varphi \in [-90^\circ, 90^\circ]$  is the latitude.

### 2.2 Haversine Distance

Client-side distance computations (circle radius estimation, road length checks) use the Haversine formula. Given two geographic coordinates  $p_1 = (\lambda_1, \varphi_1)$  and  $p_2 = (\lambda_2, \varphi_2)$ ,

the great-circle distance is:

$$d(p_1, p_2) = 2R \arctan\left(\frac{\sqrt{a}}{\sqrt{1-a}}\right) \quad (1)$$

where

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2)$$

with  $\Delta\varphi = \varphi_2 - \varphi_1$  and  $\Delta\lambda = \lambda_2 - \lambda_1$  in radians, and the mean Earth radius  $R = 6,371,000$  m (the value used throughout the codebase; see `GEOMETRY_CONFIG.earthRadiusMeters` in `src/config.ts`).

*Remark 2.1.* The Haversine formula is numerically stable for all distances, including antipodal points, unlike the simpler spherical law of cosines. PostGIS server-side geography distances use the WGS84 spheroid, which introduces a relative error below 0.3% compared to Haversine; this discrepancy is acceptable for all NARS use cases.

## 2.3 Circle Approximation

City centres are stored as (centre, radius) pairs and rendered as  $n$ -sided regular polygons. The  $k$ -th vertex of the approximating polygon is:

$$v_k = \left( \lambda_c + \frac{r}{R \cos \varphi_c} \cdot \cos\left(\frac{2\pi k}{n}\right), \varphi_c + \frac{r}{R} \cdot \sin\left(\frac{2\pi k}{n}\right) \right), \quad k = 0, \dots, n-1 \quad (3)$$

where  $(\lambda_c, \varphi_c)$  is the centre,  $r$  the radius in metres, and  $n$  the number of segments. Two values are used:  $n = 64$  for rendering (smooth appearance) and  $n = 16$  for edit mode (lighter vertex handle count).

## 2.4 Coordinate Encoding

Features are persisted as JSON objects in a PostgreSQL `text` column. The coordinates array uses the convention `{lat, lng}` (latitude-first), while GeoJSON and PostGIS use longitude-first `[λ, φ]`. All SQL fragments in `SqlFragments.cs` perform the swap explicitly:

Listing 1: Coordinate-order swap in SQL fragment

```

1 json_build_array (
2     (c->>'lng')::float,      -- longitude first (GeoJSON)
3     (c->>'lat')::float       -- latitude second
4 )

```

# 3 Feature Type Taxonomy

## 3.1 Type and Layer Hierarchy

Each persisted feature carries two discriminators: a *type* (one of eight top-level classes) and a *layer* (a subtype within that class). Table 1 enumerates the full taxonomy.

Table 1: Feature type and layer taxonomy

| Type key        | Table            | Layer keys                                                                         | Geometry       |
|-----------------|------------------|------------------------------------------------------------------------------------|----------------|
| area            | areas            | central_urban,<br>secondary_urban,<br>scattered*                                   | Polygon        |
| district        | districts        | housing_estate,<br>urban_pole, district,<br>trad_activities_zone,<br>industry_zone | Polygon        |
| city_center     | city_centers     | city_center                                                                        | Point + radius |
| road            | roads            | boulevard, avenue,<br>street, drive, lane,<br>cul_de_sac, way                      | LineString     |
| house_entrance  | house_entrances  | main_entrance,<br>secondary_entrance                                               | Point          |
| public_building | public_buildings | public_building                                                                    | Point          |
| public_space    | public_spaces    | garden, square                                                                     | Polygon        |
| naming_panel    | naming_panels    | naming_panel                                                                       | Point          |

\*Auto-computed; cannot be saved manually.

### 3.2 Feature Registry

A central `feature_registry` table maintains a (UUID, type-string) index over all eight type tables, enabling  $O(1)$  type lookup given only a feature ID. The invariant is:

$$\forall r \in \text{feature\_registry} : r.id \in \bigcup_{T \in \mathcal{T}} \pi_{id}(T) \quad (4)$$

where  $\mathcal{T}$  is the set of eight type tables and  $\pi_{id}$  denotes projection onto the ID column. This invariant is enforced by the transactional insert in `SaveFeature` and by the dynamic orphan-cleanup query in `ClearFeatures`.

### 3.3 Payload Bounds

To guard against memory exhaustion, two numeric bounds are enforced server-side:

$$|\text{rawJson}| \leq C_{\text{payload}} = 524,288 \text{ bytes} \quad (= 512 \text{ KiB}) \quad (5)$$

$$|\mathcal{C}| \leq C_{\text{coords}} = 10,000 \quad (6)$$

where  $|\mathcal{C}|$  is the cardinality of the coordinate array in a validation request. In addition, all coordinate values are checked for membership in  $\mathbb{R} \setminus (\{\text{NaN}\} \cup \{+\infty, -\infty\})$  before any PostGIS query is executed.

## 4 Geometric Validation Predicates

### 4.1 Road Validation

A road is defined as an ordered sequence of geographic coordinates  $\mathcal{R} = (p_0, p_1, \dots, p_{m-1})$  with  $m \geq 2$ . Two predicates must hold before a road is accepted.

#### 4.1.1 Angular Deviation Bound

For any three consecutive vertices  $A = p_i$ ,  $B = p_{i+1}$ ,  $C = p_{i+2}$  define the planar direction vectors (in the longitude–latitude plane):

$$\mathbf{u} = (\lambda_B - \lambda_A, \varphi_B - \varphi_A) \quad (7)$$

$$\mathbf{v} = (\lambda_C - \lambda_B, \varphi_C - \varphi_B) \quad (8)$$

The deflection angle  $\theta_i$  at vertex  $B$  is:

$$\theta_i = \arccos\left(\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \cdot \|\mathbf{v}\|_2}\right), \quad \theta_i \in [0^\circ, 180^\circ] \quad (9)$$

The road is valid if and only if:

$$\forall i \in \{0, \dots, m-3\} : \theta_i \leq \Theta_{\max} = 90^\circ \quad (10)$$

*Remark 4.1.* The dot product is clamped to  $[-1, 1]$  before the arccos to prevent floating-point errors from producing a domain exception. The check is skipped when  $\|\mathbf{u}\|_2 < 10^{-10}$  or  $\|\mathbf{v}\|_2 < 10^{-10}$  (coincident vertices), which is correct as degenerate segments carry no directional information.

#### 4.1.2 Network Connectivity

A new road  $R_{\text{new}}$  must spatially connect to the existing road network  $\mathcal{N}$  (unless  $\mathcal{N} = \emptyset$ , i.e. this is the first road). Connectivity is defined as:

$$\exists R \in \mathcal{N} : d_{\text{geo}}(R_{\text{new}}, R) \leq \delta_{\text{road}} = 20 \text{ m} \quad (11)$$

where  $d_{\text{geo}}$  is the spheroidal distance computed by PostGIS `ST_DWithin` on geography columns, which uses the WGS84 ellipsoid.

### 4.2 District Validation

A district is a polygon  $D_{\text{new}}$ . Three conditions are checked.

#### 4.2.1 Non-overlap

The new district must not overlap any existing district owned by the same user:

$$\forall D \in \mathcal{D} : \neg \text{ST\_Intersects}(D_{\text{new}}, D) \quad (12)$$

Note that PostGIS `ST_Intersects` returns `true` for containment, which is more conservative than `ST_Overlaps` (which requires interiors to intersect but neither to contain the other). This choice is deliberate: a district entirely inside another is semantically incorrect.

### 4.2.2 Urban-Area Containment

Let  $\mathcal{U}$  be the set of urban area polygons (layers `central_urban` and `secondary_urban`). The *sibling count* of  $D_{\text{new}}$  within a given urban area  $U \in \mathcal{U}$  is:

$$\sigma(D_{\text{new}}, U) = |\{D \in \mathcal{D} \mid \text{ST\_Intersects}(U, D_{\text{new}}) \wedge \text{ST\_Intersects}(U, D)\}| \quad (13)$$

### 4.2.3 Boundary Adjacency

If  $\sigma(D_{\text{new}}, U) > 0$  (i.e. at least one sibling district already exists in the same urban area), then  $D_{\text{new}}$  must share a boundary segment with one of its siblings:

$$\exists D \in \mathcal{D} : \text{ST\_Touches}(D_{\text{new}}, D) \vee \text{ST\_Intersects}(\partial D_{\text{new}}, \partial D) \quad (14)$$

where  $\partial D$  denotes the topological boundary (a closed linestring). The redundant intersection of boundaries is included to handle the edge case where two polygons share a collinear sub-segment that `ST_Touches` (which requires the interiors to be disjoint) might miss due to floating-point rounding.

**Exemptions.** Districts of type `trad_activities_zone` and `industry_zone` are exempt from adjacency checking, as these zones may appear in scattered (non-contiguous) areas.

## 4.3 District Coverage

After all districts have been drawn, the operator may request a coverage check. Let  $U$  be the union of all urban area polygons and  $\hat{D}$  the union of all district polygons. The coverage condition is:

$$\text{ST\_Covers}\left(\text{ST\_Buffer}\left(\hat{D}, \varepsilon_{\text{cov}}\right), U\right) = \text{true} \quad (15)$$

where the buffer tolerance is  $\varepsilon_{\text{cov}} = 10$  m. This tolerance accounts for digitising imprecision at shared boundaries. `ST_Buffer` here operates on the geography type, so the 10 m is a true metric distance on the ellipsoid.

**Theorem 4.1** (Coverage completeness). *Let  $U$  and  $\hat{D}$  be valid PostGIS geometry objects with  $\hat{D} \subseteq \text{ST\_Buffer}(U, \varepsilon)$  for some  $\varepsilon \geq 0$ . Then eq. (15) holds if and only if every point  $q \in U$  has a corresponding point  $q' \in \hat{D}$  with  $d_{\text{geo}}(q, q') \leq \varepsilon_{\text{cov}}$ .*

*Remark 4.2.* In practice the tolerance is never exercised on correctly drawn data; it exists solely to absorb the  $\sim 1$  m snapping imprecision that accumulates when the frontend rounds coordinate pairs to `float64`.

## 4.4 Main Urban Area Existence

Before any district or road may be created, at least one area of layer `central_urban` must exist:

$$|\{A \in \text{areas} \mid A.\text{layer} = \text{central\_urban} \wedge A.\text{user\_id} = u\}| \geq 1 \quad (16)$$

## 5 Snap Geometry

### 5.1 Coordinate Spaces

NARS uses two coordinate spaces simultaneously during drawing:

- *Geographic space*  $\mathcal{G} \subset \mathbb{R}^2$ : longitude–latitude pairs.
- *Screen space*  $\mathcal{S} \subset \mathbb{R}^2$ : pixel coordinates relative to the MapLibre canvas origin, with  $y$ -axis pointing downward.

The MapLibre `map.project{lngLat})` function provides the continuous bijection  $\Pi : \mathcal{G} \rightarrow \mathcal{S}$  (zoom-level dependent). Snap distances are measured in screen space because a fixed pixel threshold corresponds to a consistent physical pointing precision regardless of zoom level.

### 5.2 Snap Threshold Parameters

Table 2: Snap threshold values (pixels) per snap type

| Snap type     | $\delta_{\text{snap}}$ (px) | Target geometry                             |
|---------------|-----------------------------|---------------------------------------------|
| Vertex snap   | 40                          | Existing polygon/polyline vertices          |
| Edge snap     | 40                          | Interior of existing polygon/polyline edges |
| Circle snap   | 20                          | Approximated circle ring vertices           |
| Midpoint snap | 20                          | Edge midpoints                              |

### 5.3 Vertex Snap

Given cursor position  $\mathbf{q} \in \mathcal{S}$  and a set of candidate vertices  $V = \{\Pi(p_k)\}_{k=1}^N$  in screen space, the snap target is:

$$\mathbf{q}^* = \arg \min_{v \in V} \|\mathbf{v} - \mathbf{q}\|_2, \quad \text{if } \min_{v \in V} \|\mathbf{v} - \mathbf{q}\|_2 \leq \delta_{\text{vertex}} \quad (17)$$

If no vertex satisfies the threshold, no snap is applied.

### 5.4 Edge Snap

For an edge defined by screen-space endpoints  $\mathbf{a}, \mathbf{b} \in \mathcal{S}$ , the nearest point on the segment to cursor  $\mathbf{q}$  is:

$$\mathbf{e}^* = \mathbf{a} + t^*(\mathbf{b} - \mathbf{a}), \quad t^* = \text{clamp}\left(\frac{(\mathbf{q} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})}{\|\mathbf{b} - \mathbf{a}\|_2^2}, 0, 1\right) \quad (18)$$

The edge snap fires when  $\|\mathbf{q} - \mathbf{e}^*\|_2 \leq \delta_{\text{edge}}$  and the snapped geographic position is  $\Pi^{-1}(\mathbf{e}^*) = \text{map.unproject}(\mathbf{e}^*)$ .

## 5.5 Phase-Specific Snap Targets

Not every drawing phase snaps to every existing feature layer. The snap-target relation  $\mathcal{T} \subseteq \text{Phase} \times \text{Layer}$  is defined in `SNAP_CONFIG.snapTargets`:

$$\mathcal{T} = \begin{cases} \text{areas} & \mapsto \{\text{areas}\} \\ \text{districts} & \mapsto \{\text{areas}, \text{districts}\} \\ \text{roads} & \mapsto \{\text{areas}, \text{cityCenter}, \text{roads}\} \\ \text{publicBuildings} & \mapsto \{\text{areas}, \text{publicBuildings}, \text{publicSpaces}\} \\ \text{publicSpaces} & \mapsto \{\text{areas}, \text{publicBuildings}, \text{publicSpaces}\} \\ \text{others} & \mapsto \emptyset \end{cases} \quad (19)$$

## 5.6 Marker-Pointer Interception

In edit mode, Geoman’s internal `markerPointer.marker` calls `setLngLat(rawMousePos)` in the bubble phase of the mouse-move event, after any capture-phase listener would have set a snapped position. To prevent the overwrite, NARS monkey-patches `setLngLat`:

$$\widehat{\text{setLngLat}}(\mathbf{q}_{\text{raw}}) = \text{setLngLat} \left( \begin{cases} \Pi^{-1}(\mathbf{q}_{\text{snap}}^*) & \text{if } \mathbf{q}_{\text{snap}}^* \text{ exists} \\ \mathbf{q}_{\text{raw}} & \text{otherwise} \end{cases} \right) \quad (20)$$

The patch is installed in `enableEditMode` and removed in `disableEditMode`, leaving the original function pointer intact via the closure variable `_origSetLngLat`.

# 6 Scattered Area Computation

Urban areas may be of three subtypes: `central_urban`, `secondary_urban`, and `scattered`. The third subtype is never drawn manually; it is auto-computed as the set-theoretic complement of the explicit urban areas within the commune boundary.

**Definition 6.1** (Scattered area). Let  $B$  be the commune boundary polygon and let  $U = \bigcup_{A \in \mathcal{A}_{\text{urban}}} A$  be the union of all explicitly drawn urban area polygons for a given user. The scattered area is:

$$S = B \setminus U = B \cap U^c \quad (21)$$

In PostGIS terms, this is computed as `ST_Difference(B, ST_Union( $\mathcal{A}$ ))`. The result may be a `MULTIPOLYGON` if the complement is disconnected. Each connected component is stored as a separate `area` record with layer `scattered`.

## 6.1 Trigger and Background Execution

The scattered-area computation is triggered by any mutation to the `areas` table: save, update, or delete. It is dispatched to an `IBackgroundTaskQueue` (a `Channel<Func<IServiceProvider, CancellationToken, ValueTask>>`) so it does not block the HTTP response. The queue is bounded and processes work items sequentially:

$$\text{throughput} = \frac{1}{\tau_{\text{PostGIS}}} \quad (22)$$

where  $\tau_{\text{PostGIS}}$  is the wall-clock time of the `ST_Difference / ST_Union` query. For typical commune geometries ( $|\mathcal{A}| \lesssim 10$ , coordinate count  $\lesssim 500$ ),  $\tau_{\text{PostGIS}} < 50$  ms.

## 7 Security Primitives

### 7.1 Password Storage: BCrypt

User passwords are stored as BCrypt digests. For a password  $\pi$  and salt  $s$  of length 128 bits (16 bytes), BCrypt computes:

$$H_{\text{pwd}} = \text{bcrypt}(\pi, s, \text{cost} = 11) \quad (23)$$

The BCrypt work factor 11 implies  $2^{11} = 2,048$  iterations of the Blowfish key schedule. The output is a 60-character string embedding the algorithm version, cost, salt, and hash. Verification is performed by `BCrypt.Net.BCrypt.Verify( $\pi$ ,  $H_{\text{pwd}}$ )`, which re-derives the hash from the embedded salt and compares in constant time.

### 7.2 JWT Access Token

JSON Web Tokens are signed with HMAC-SHA256. For header  $h$ , claims payload  $c$ , and secret key  $k$  (a UTF-8 string of at least 32 characters):

$$\text{JWT} = \underbrace{\text{Base64url}(h)}_{\text{header}} . \underbrace{\text{Base64url}(c)}_{\text{payload}} . \underbrace{\text{Base64url}(\text{HMAC-SHA-256}(k, h.c))}_{\text{signature}} \quad (24)$$

The claims set embedded in each token is:

$$c = \{\text{user\_id}, \text{username}, \text{name}, \text{email}, \text{commune\_id}, \text{exp} = t_{\text{now}} + T_{\text{access}}\} \quad (25)$$

where the access-token lifetime is  $T_{\text{access}} = 1,440 \text{ min} = 24 \text{ h}$  (configurable via `Jwt:ExpiresInMinutes`).

The token is delivered exclusively via an `HttpOnly; SameSite=Lax; Path=/` cookie, never in a response body field that JavaScript could persist to `localStorage`.

### 7.3 Refresh Token Protocol

Refresh tokens implement a rotation scheme. A raw token  $\rho$  of 512 bits (64 bytes) is generated by a cryptographically secure PRNG:

$$\rho \xleftarrow{\$} \{0, 1\}^{512}, \quad H_{\rho} = \text{SHA-256}(\rho) \quad (26)$$

The server stores  $H_{\rho}$  (never  $\rho$ ); the client stores  $\rho$  in an `HttpOnly` cookie. On each refresh:

1. A PostgreSQL `SELECT ...FOR UPDATE SKIP LOCKED` serialises concurrent refresh attempts on the same token row.
2. The stored record is marked `revoked = true`.
3. A new pair  $(\rho', H_{\rho'})$  is generated and stored.
4. A new JWT and the new  $\rho'$  cookie are issued.

This protocol provides *refresh token rotation* (RTR): any attempt to reuse a revoked token (e.g. from a stolen cookie) returns `401 Unauthorized`.

## 7.4 Account Lockout

After  $F_{\max} = 5$  consecutive failed login attempts for a given username, the account is locked for  $T_{\text{lock}} = 30$  min:

$$\text{LockedUntil} = \begin{cases} t_{\text{now}} + T_{\text{lock}} & \text{if } F \geq F_{\max} \\ \text{null} & \text{otherwise} \end{cases} \quad (27)$$

A successful login atomically resets the counter:  $F \leftarrow 0$ ,  $\text{LockedUntil} \leftarrow \text{null}$ .

## 7.5 CSRF Protection

CSRF protection covers server-rendered form submissions (the login page). The ASP.NET Core `IAntiforgery` service uses the *double-submit cookie* pattern:

$$\text{Valid} \iff \text{cookie}[\text{X-CSRF-TOKEN-COOKIE}] \neq \emptyset \wedge \text{header}[\text{X-CSRF-Token}] = \mathcal{F}(\text{cookie}[\text{X-CSRF-TOKEN}]) \quad (28)$$

where  $\mathcal{F}$  is the ASP.NET Core antiforgery token derivation function (internally HMAC-based). API endpoints (`/api/...`) are exempt because all auth cookies carry `SameSite=Lax`, which prevents cross-origin POST/PUT/DELETE requests from attaching session cookies.

## 8 Rate Limiting

NARS uses ASP.NET Core’s sliding-window rate limiter. Two policies are defined; table 3 gives the default configuration values from `appsettings.json`.

Table 3: Rate-limiting policy parameters

| Policy | $P$ (permits) | $W$ (window) | $S$ (segments) | Scope                                                                        |
|--------|---------------|--------------|----------------|------------------------------------------------------------------------------|
| auth   | 5             | 30 s         | 3              | POST <code>/api/signin</code> , <code>/signup</code> , <code>/refresh</code> |
| clear  | 3             | 10 min       | 1              | POST <code>/api/clear</code>                                                 |
| api    | 60            | 1 min        | 6              | All other authenticated endpoints                                            |

For a sliding-window limiter with permit budget  $P$ , window duration  $W$ , and  $S$  segments, the effective replenishment rate is:

$$r = \frac{P}{W} \text{ req/s} \quad (29)$$

At each segment boundary  $t_s = k \cdot W/S$  ( $k \in \mathbb{N}$ ), requests from the oldest segment are evicted from the count. A request at time  $t$  is accepted if and only if:

$$\sum_{i=0}^{S-1} c\left(t - \frac{iW}{S}, t - \frac{(i+1)W}{S}\right) < P \quad (30)$$

where  $c(t_1, t_2)$  counts accepted requests in the half-open interval  $(t_1, t_2]$ .

For the **auth** policy, the maximum burst is 5 requests in any 30-second window, segmented into 10-second buckets. This provides protection against both slow-rate credential-stuffing attacks and rapid brute-force bursts, with the lockout mechanism (section 7.4) providing a secondary defence layer.

## 9 REST API Summary

### 9.1 Endpoint Taxonomy

Table 4 enumerates the complete REST surface. All endpoints under `/api/` require HTTP cookie authentication (the `access_token` cookie) except `/api/signin`, `/api/signup`, `/api/refresh`, and `/health`.

Table 4: REST API endpoints

| Method                | Path                                 | Description                  | Rate policy |
|-----------------------|--------------------------------------|------------------------------|-------------|
| <i>Authentication</i> |                                      |                              |             |
| POST                  | /api/signup                          | Register new user            | auth        |
| POST                  | /api/signin                          | Authenticate, issue tokens   | auth        |
| POST                  | /api/logout                          | Revoke re-fresh token        | api         |
| POST                  | /api/refresh                         | Rotate re-fresh token        | auth        |
| GET                   | /api/current_user                    | Return authenticated profile | api         |
| <i>Features</i>       |                                      |                              |             |
| POST                  | /api/save                            | Persist new feature          | api         |
| GET                   | /api/load                            | Paginated feature list       | api         |
| PUT                   | /api/update/:id                      | Update geometry or label     | api         |
| DELETE                | /api/delete/:id                      | Delete feature               | api         |
| POST                  | /api/clear                           | Delete all features          | clear       |
| GET                   | /api/stats                           | Per-type feature counts      | api         |
| <i>Validation</i>     |                                      |                              |             |
| GET                   | /api/validate/area/main-urban-exists | Urban area existence check   | api         |
| POST                  | /api/validate/road                   | Road angle + connectivity    | api         |
| POST                  | /api/validate/district               | District overlap + adjacency | api         |
| GET                   | /api/validate/districts/coverage     | Full coverage check          | api         |
| <i>Auxiliary</i>      |                                      |                              |             |
| GET                   | /api/locations                       | Commune/Daira/Wilaya lookup  | api         |
| GET                   | /api/spatial/commune-boundary        | Commune boundary polygon     | api         |
| GET                   | /api/scattered-status                | Background task error status | api         |
| GET                   | /health                              | Database connectivity probe  | —           |

## 9.2 Pagination Model

The `/api/load` endpoint uses offset-based pagination:

$$\text{take} = \text{clamp}(\text{take}_{\text{req}}, 1, 2,000) \quad (31)$$

The response body carries both the data page and the total count (computed in the same CTE query via a `COUNT(*)` window), enabling clients to render pagination controls without an additional round-trip.

## 9.3 Feature Query Architecture

All feature types are queried via a single `UNION ALL` CTE, projecting a common 7-column schema:

Listing 2: `UNION ALL` feature query skeleton

```
1 WITH all_features AS (  
2     SELECT id, user_id, label, data, created_at, layer,  
3           'area' AS feature_type FROM areas  
4     UNION ALL  
5     SELECT id, user_id, label, data, created_at, layer,  
6           'district' FROM districts  
7     -- ... 6 more type tables  
8 ),  
9 filtered AS (  
10    SELECT * FROM all_features WHERE user_id = @uid  
11 ),  
12 total AS (SELECT COUNT(*) AS total_count FROM filtered)  
13 SELECT f.*, t.total_count  
14 FROM filtered f, total t  
15 ORDER BY f.created_at  
16 OFFSET @skip LIMIT @take;
```

The query is expressed as a verbatim string constant (no string interpolation) in `FeatureQueryHelper.cs`, eliminating any risk of SQL injection through query construction.

# 10 System Architecture

## 10.1 Layered Component Model

Figure 2 presents the four-layer architecture of NARS.

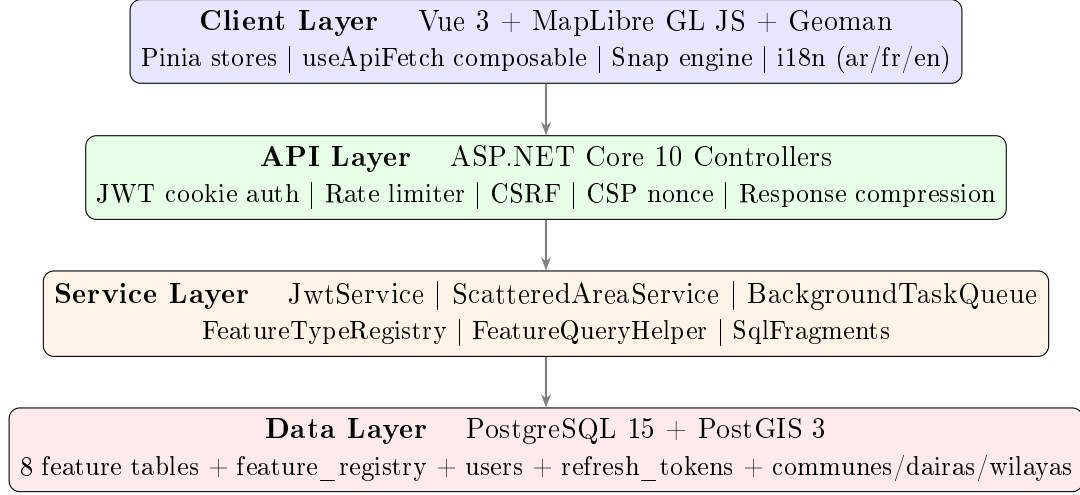


Figure 2: Four-layer architecture of NARS.

## 10.2 Technology Stack

Table 5: Technology stack summary

| Component            | Technology                      | Version   |
|----------------------|---------------------------------|-----------|
| Backend runtime      | .NET (ASP.NET Core)             | 10.0      |
| ORM / query          | Entity Framework Core + ADO.NET | 9.x       |
| Database             | PostgreSQL + PostGIS            | 15 / 3.x  |
| Password hashing     | BCrypt.Net-Next                 | —         |
| JWT library          | System.IdentityModel.Tokens.Jwt | —         |
| Frontend framework   | Vue 3 (Composition API)         | 3.5.x     |
| State management     | Pinia                           | 3.x       |
| Map engine           | MapLibre GL JS                  | 5.x       |
| Draw/edit layer      | Geoman for MapLibre (free)      | 0.7.x     |
| Spatial utilities    | Turf.js                         | 7.x       |
| Build tool           | Vite + vue-tsc                  | 8.x / 3.x |
| XSS sanitiser        | DOMPurify                       | 3.x       |
| Internationalisation | vue-i18n                        | 11.x      |

## 11 Data Flow: Feature Save Transaction

The save of a new feature involves a two-phase database transaction to maintain the registry invariant (eq. (4)).

---

**Algorithm 1** SaveFeature transactional protocol

---

**Require:** type  $\tau$ , layer  $\ell$ , label  $L$ , data  $D$ , user  $u$ , commune  $c$

**Ensure:** registry invariant (4) holds after commit

```
1: Validate  $\tau \in \mathcal{T}$  and  $(\tau, \ell)$  is a valid type-layer pair
2: Validate  $\tau \neq \text{area.scattered}$  ▷ auto-computed only
3: Validate  $|D_{\text{raw}}| \leq C_{\text{payload}}$  ▷ eq. 5
4: if  $\tau = \text{house\_entrance}$  and  $\ell = \text{main\_entrance}$  then
5:   Extract roadDbId from  $D$ ; verify road exists and belongs to  $u$ 
6: end if
7:  $id \leftarrow \text{Guid.CreateVersion7}()$  ▷ UUID v7: monotonically sortable
8:  $e \leftarrow \text{FeatureTypeRegistry.CreateEntity}(\tau, id, u, \ell, L, D)$ 
9: BEGIN transaction  $T$ 
10: Insert  $e$  into the appropriate type table
11: Insert  $(id, \tau)$  into feature_registry
12: COMMIT  $T$ 
13: if  $\tau = \text{area}$  then
14:   Enqueue scattered-area background refresh
15: end if
16: return  $id$ 
```

---

The use of UUID v7 (time-ordered) as the primary key ensures that the default `ORDER BY created_at` in the load query is equivalent to `ORDER BY id`, allowing future index optimisation.

## 12 Internationalisation Model

NARS supports three locales: English (`en`), French (`fr`), and Arabic (`ar`). The locale catalogue is a JSON dictionary  $\mathcal{L} : \text{Key} \rightarrow \text{String}$  loaded by `vue-i18n`. Arabic activates right-to-left layout via a CSS `dir="rtl"` attribute.

Administrative names are stored bilingual in the database (`commune_fr/commune_ar`, `wilaya_fr/wilaya_ar`, etc.) and the API returns both variants in every response, letting the client select the appropriate column without an additional round-trip.

## 13 Circle Ring: Spherical Direct Problem

The naïve formula in eq. (3) approximates the vertex longitude offset with a flat-Earth cosine scale. The production implementation in `geometry.ts` solves the exact *spherical direct problem*: given centre  $(\varphi_c, \lambda_c)$ , angular distance  $\alpha = r/R$ , and bearing  $\beta_k = 2\pi k/n$ , the  $k$ -th vertex is:

$$\varphi_k = \arcsin(\sin \varphi_c \cos \alpha + \cos \varphi_c \sin \alpha \cos \beta_k) \quad (32)$$

$$\lambda_k = \lambda_c + \arctan 2\left(\sin \beta_k \sin \alpha \cos \varphi_c, \cos \alpha - \sin \varphi_c \sin \varphi_k\right) \quad (33)$$

where all angles are in radians and coordinates are converted back to degrees after computation. This is equivalent to Vincenty's direct formula on a sphere and introduces

no first-order approximation error in the latitude direction. The naïve formula (3) is retained in the documentation for pedagogical clarity; (32)–(33) are what actually execute.

## 14 Road Direction Algorithm

Road direction (start  $\rightarrow$  end orientation) is required for rendering directional arrows and for correctly sequencing house-entrance numbers (section 15). The algorithm in `road-directions.ts` runs in five phases.

### 14.1 Phase 1 — Topology Graph Construction

Let  $\mathcal{R} = \{R_1, \dots, R_n\}$  be the set of road polylines. Each road  $R_i$  is an ordered list of coordinates  $R_i = (p_0^{(i)}, \dots, p_{m_i-1}^{(i)})$ . Define the *junction threshold*  $\delta_J = 30$  m.

For every ordered pair  $(R_i, R_j)$  with  $i \neq j$ , two junction types are detected:

**T-junction.** An endpoint  $p \in \{p_0^{(j)}, p_{m_j-1}^{(j)}\}$  that satisfies  $d_{\text{geo}}(p, p_0^{(i)}) > \delta_J$  and  $d_{\text{geo}}(p, p_{m_i-1}^{(i)}) > \delta_J$ , yet whose nearest point on the body of  $R_i$  is within  $\delta_J$ :

$$q^* = \text{proj}_{R_i}(p), \quad d_{\text{geo}}(p, q^*) \leq \delta_J \quad (34)$$

**X-junction.** Any geometric intersection point of lines  $R_i$  and  $R_j$  that lies strictly in the interior of both (not within  $\delta_J$  of either road’s endpoints).

The junctions on  $R_i$  are sorted by arc-length parameter, deduplicated (any two within  $\delta_J$  are merged), and used to split  $R_i$  into sub-segments  $S_{i,1}, \dots, S_{i,k_i}$ . Each sub-segment becomes an undirected edge in the topology graph  $G = (V, E)$ , where vertices are shared at all junction points (merged within  $\delta_J$ ).

**Definition 14.1** (Topology graph).  $G = (V, E)$  where  $V = \{v \in \mathcal{G} \mid v \text{ is an endpoint or junction point}\}$  and  $e = (v_a, v_b, s) \in E$  iff sub-segment  $s$  connects  $v_a$  to  $v_b$ . Nodes within  $\delta_J$  of each other are unified.

### 14.2 Phase 2A — City-Centre Orientation (DFS)

When at least one city centre exists, a depth-first search from the city-centre perimeter assigns orientation to every reachable segment.

Let  $\mathcal{C} = (c, r)$  be a city centre (centre  $c$ , radius  $r$ ). The *seed nodes* are:

$$V_{\text{seed}} = \{v \in V \mid |d_{\text{geo}}(c, v) - r| \leq \delta_J\} \quad (35)$$

For each seed  $v_s \in V_{\text{seed}}$ , `ORIENTFROM`( $v_s$ ) is called. At each recursive call with *fromNode*  $v_f$ :

1. Collect unvisited, unassigned edges incident to  $v_f$ , sorted by ascending distance of their far endpoint to  $c$  (closest roads oriented first).
2. For each edge  $e = (v_f, v_{to}, s)$ :

$$s.\text{reversed} = [d_{\text{geo}}(s_0, v_f) > d_{\text{geo}}(s_{|s|-1}, v_f)] \quad (36)$$

where  $s_0$  and  $s_{|s|-1}$  are the first and last coordinates of  $s$ .

3. Mark  $e$  and all segments of the same road as visited.
4. Recurse: `ORIENTFROM`( $v_{to}$ ).

### 14.3 Phase 2B — Geographic Fallback

For roads unreached by the DFS (or when no city centre exists), orientation is assigned by cardinal direction:

$$s.\text{reversed} = \begin{cases} \varphi_0 < \varphi_{|\cdot|-1} & \text{if } |\Delta\varphi| \geq |\Delta\lambda| \quad (\text{N-S road: start at north}) \\ \lambda_0 < \lambda_{|\cdot|-1} & \text{otherwise} \quad (\text{E-W road: start at east}) \end{cases} \quad (37)$$

### 14.4 Phase 3 — Majority Vote

Sub-segments of the same road may have received different orientations via different DFS paths. The final orientation of road  $R_i$  is determined by majority vote over its  $k_i$  sub-segments:

$$R_i.\text{reversed} = \left[ \sum_{j=1}^{k_i} \mathbf{1}[S_{i,j}.\text{reversed}] > \frac{k_i}{2} \right] \quad (38)$$

### 14.5 Phase 4 — Dead-End Correction

Let  $\deg(v)$  be the degree of node  $v$  in  $G$ . If a road's first endpoint  $v_a$  has  $\deg(v_a) = 1$  and its last endpoint  $v_b$  has  $\deg(v_b) > 1$ , the connected side ( $v_b$ ) must be the *from* side, overriding the majority vote:

$$R_i.\text{reversed} = \neg [\deg(v_a) > \deg(v_b)] \quad (39)$$

When both endpoints have  $\deg = 1$  (road split by a T-junction on its body), the from-side is the endpoint closer to the city centre.

### 14.6 Segment Bearing

The directional arrow angle  $\theta$  for endpoint markers is computed in MapLibre screen space:

$$\theta = \arctan 2(\Pi(p_{i+1})_y - \Pi(p_i)_y, \Pi(p_{i+1})_x - \Pi(p_i)_x) \cdot \frac{180}{\pi} \quad (40)$$

where  $\Pi$  is the MapLibre `project` function at the current zoom level.

## 15 House Entrance Numbering

House entrances connected to a reference road are assigned numbers by projecting each unassigned entrance onto the road's polyline, sorting by arc-length, and allocating odd numbers to the left side and even numbers to the right side.

### 15.1 Arc-Length Projection

For a road  $R = (p_0, \dots, p_{m-1})$  (as a Turf.js `LineString`) and an entrance point  $e$ , the *arc-length parameter*  $\ell(e) \geq 0$  is the cumulative metric distance from  $p_0$  to the nearest point on  $R$ :

$$e^* = \text{NearestPointOnLine}(R, e), \quad \ell(e) = \text{snapped.properties.location} \quad (41)$$

computed by Turf.js `nearestPointOnLine` with `units="meters"`.

## 15.2 Numbering Assignment

Let  $\mathcal{E}_u$  be the set of unassigned entrances projected and sorted by arc-length:  $e_1, e_2, \dots, e_k$  with  $\ell(e_1) \leq \ell(e_2) \leq \dots \leq \ell(e_k)$ .

The seed counters are initialised from already-assigned entrances on the same road:

$$o_0 = \max(\{n \mid n \text{ odd, } n \text{ assigned to a left entrance on } R\} \cup \{-1\}) + 2 \quad (42)$$

$$e_0 = \max(\{n \mid n \text{ even, } n \text{ assigned to a right entrance on } R\} \cup \{0\}) + 2 \quad (43)$$

The number assigned to the  $i$ -th unassigned entrance is:

$$N(e_i) = \begin{cases} o_j \leftarrow o_{j-1} + 2 & \text{if } e_i.\text{side} = \text{left} \\ e_j \leftarrow e_{j-1} + 2 & \text{if } e_i.\text{side} = \text{right} \end{cases} \quad (44)$$

where  $j$  increments independently for odd and even counters, ensuring no two left-side (or right-side) entrances share a number.

This scheme is consistent with the international convention of odd numbers on one side of a street and even on the other, sequenced in order of increasing distance from the origin of the road.

## 16 Point-in-Polygon: Ray-Casting Algorithm

Two spatial hit-tests use a client-side even-odd rule rather than a PostGIS query: (1) checking whether a new feature falls within the municipality boundary, and (2) checking whether a point lies in the scattered area. Both use the same `pointInRing` function from `geometry.ts`.

**Definition 16.1** (Even-odd rule). A point  $q = (\varphi, \lambda)$  lies *inside* polygon ring  $\mathcal{P} = (v_0, v_1, \dots, v_{n-1})$  (with  $v_n = v_0$ ) if and only if the number of ring edges that cross the horizontal ray emanating rightward from  $q$  is odd:

$$\mathbf{1}[q \in \mathcal{P}] = \left( \sum_{i=0}^{n-1} \mathbf{1}[\text{edge } (v_i, v_{i+1}) \text{ crosses the ray}] \right) \bmod 2 = 1 \quad (45)$$

An edge  $(v_i, v_{i+1})$  with  $v_i = (\varphi_i, \lambda_i)$  crosses the horizontal ray from  $(\varphi, \lambda)$  when:

$$[(\lambda_i > \lambda) \neq (\lambda_{i+1} > \lambda)] \wedge \left[ \varphi < \frac{(\varphi_{i+1} - \varphi_i)(\lambda - \lambda_i)}{\lambda_{i+1} - \lambda_i} + \varphi_i \right] \quad (46)$$

The implementation toggles an `inside` boolean for each crossing, producing  $O(n)$  time and  $O(1)$  space complexity.

For the scattered area (section 6), which may contain holes, the predicate is extended:

$$q \in S \iff \text{pointInRing}(q, S.\text{outer}) \wedge \forall H \in S.\text{holes} : \neg \text{pointInRing}(q, H) \quad (47)$$

## 17 Undo Stack

NARS implements a *delete-only* undo model: the undo stack captures deleted features; vertex-level undo during drawing is delegated to Geoman's built-in right-click remove.

**Definition 17.1** (Undo stack). The undo stack  $\mathcal{U}$  is a last-in first-out sequence of records:

$$\mathcal{U} = \langle (e_1, k_1), (e_2, k_2), \dots, (e_m, k_m) \rangle \quad (48)$$

where  $e_i \in \text{LayerEntry}$  is the deleted feature's in-memory representation and  $k_i \in \text{Phase}$  is its phase key.

### 17.1 Record

Before a delete request is dispatched to the API, the entry is pushed:

$$\mathcal{U} \leftarrow \mathcal{U} \cdot \langle (e, k) \rangle \quad (49)$$

### 17.2 Restore

On Ctrl+Z, the stack is popped and the entry is re-saved via `POST /api/save`. This allocates a *new* UUID for the restored feature:

$$\text{id}_{\text{new}} \neq \text{id}_{\text{old}} \quad (50)$$

This has an important cross-reference consequence: any `secondary_entrance` records that referenced the old main-entrance ID via `mainEntranceDbId` must be updated. The restore procedure performs this repair in-memory:

$$\forall e' \in \text{houseEntrances} : e'.\text{mainEntranceDbId} = \text{id}_{\text{old}} \implies e'.\text{mainEntranceDbId} \leftarrow \text{id}_{\text{new}} \quad (51)$$

*Remark 17.1.* The undo stack lives only in browser memory and is cleared on page reload. It provides a best-effort safety net for accidental deletions within a single session, not a durable audit trail.

## 18 Client-Side Validation Pipeline

Before dispatching to the server, the frontend performs two lightweight checks that catch the most common user errors without a network round-trip.

### 18.1 Minimum Road Length

A road is rejected on the client if its total metric length is below the configured minimum:

$$\ell_{\text{road}} = \sum_{i=0}^{m-2} d_{\text{geo}}(p_i, p_{i+1}) < L_{\text{min}} = 10 \text{ m} \quad (52)$$

Length is computed by Turf.js `length(line, {units: "meters"})`, which accumulates the Haversine distance over each segment.

## 18.2 Polygon Ring Closure

The PostGIS `ST_GeomFromText` function requires the first and last vertices of a linear ring to be identical. The client enforces this before serialisation:

$$\text{If } p_0 \neq p_{m-1} : \quad \mathcal{C} \leftarrow \mathcal{C} \cup \{p_0\} \quad (53)$$

where equality is tested coordinate-wise (exact floating-point comparison on rounded `float64` values; no epsilon tolerance is needed because the first vertex is appended verbatim).

## 18.3 Two-Stage Validation Architecture

Figure 3 summarises the overall two-stage structure: client-side fast checks followed by server-side PostGIS spatial predicates.

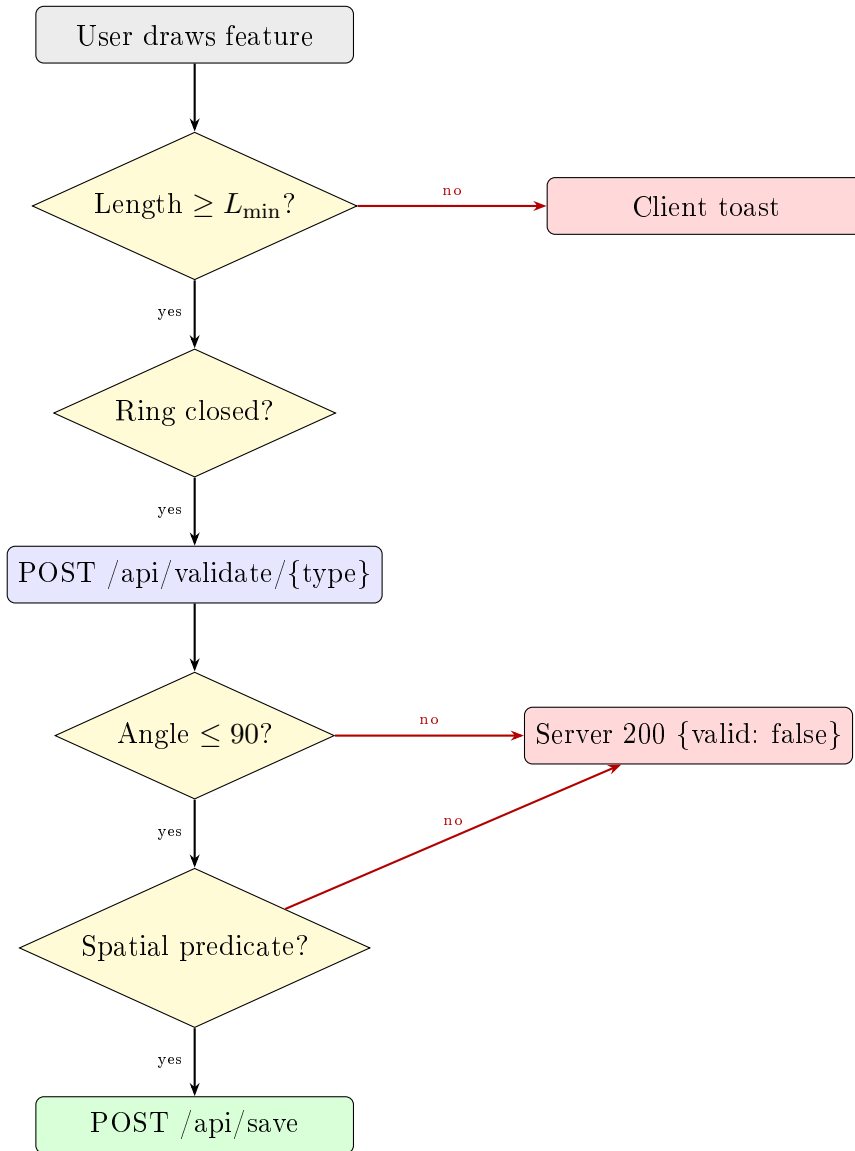


Figure 3: Two-stage validation flow for road and district features. Client-side checks (yellow) prevent trivial invalid inputs without a network round-trip. Server-side PostGIS checks enforce spatial correctness.

## 19 Database Schema

### 19.1 Relation Definitions

The database comprises 14 tables. Table 6 summarises the primary keys, foreign keys, and key index definitions as derived from `AppDbContext.cs` and the EF Core model configuration.

Table 6: Database schema summary

| Table               | PK               | FK                         | Notable indices / constraints                                    |
|---------------------|------------------|----------------------------|------------------------------------------------------------------|
| users               | id (UUID v7)     |                            | UNIQUE(email),<br>UNIQUE(username)                               |
| refresh_tokens      | id (UUID)        | user_id → users            | ix_refresh_tokens_tok                                            |
| wilayas             | wilaya_id (int)  |                            | Static reference table                                           |
| dairas              | daira_id (int)   | wilaya_id                  |                                                                  |
| communes            | commune_id (int) | daira_id                   |                                                                  |
| communes_boundaries | commune_id (int) |                            | GIST index on geometry                                           |
| feature_registry    | id (UUID v7)     |                            | feature_type string                                              |
| areas               | id (UUID v7)     |                            | ix_areas_user_id,<br>ix_areas_user_layer                         |
| districts           | id (UUID v7)     |                            | ix_districts_user_id                                             |
| city_centers        | id (UUID v7)     |                            | ix_city_centers_user_                                            |
| roads               | id (UUID v7)     |                            | ix_roads_user_id,<br>ix_roads_user_layer                         |
| house_entrances     | id (UUID v7)     | road_id → roads (nullable) | ix_house_entrances_ro<br>(partial,<br>road_id<br>IS NOT<br>NULL) |
| public_buildings    | id (UUID v7)     |                            | ix_public_buildings_u                                            |
| public_spaces       | id (UUID v7)     |                            | ix_public_spaces_user                                            |
| naming_panels       | id (UUID v7)     |                            | ix_naming_panels_user                                            |

### 19.2 Feature Base Schema

All eight feature tables share the same base schema (from `FeatureBase.cs`):

$$\text{feature} = ( \underbrace{\text{id}}_{\text{UUID v7}}, \text{user\_id}, \text{label}, \underbrace{\text{data}}_{\text{JSON text}}, \text{layer}, \text{created\_at}, \text{updated\_at} ) \quad (54)$$

Geometry is *not* stored in a native PostGIS column; it is embedded as a JSON array of `{lat, lng}` objects within `data`. This design prioritises schema flexibility over spatial index coverage. All PostGIS predicates reconstruct geometry from the JSON at query time via the `SqlFragments.PolygonFromData` and `LineStringFromData` fragments.

### 19.3 UUID v7 Temporal Ordering

Primary keys are generated client-side using `Guid.CreateVersion7()`, which encodes a Unix millisecond timestamp in the high 48 bits of the UUID:

$$\text{UUID v7} = \underbrace{48 \text{ bits: ms timestamp}}_{\text{monotonic}} \parallel \underbrace{4 \text{ bits: version} = 7}_{\text{}} \parallel \underbrace{12 \text{ bits: seq counter}}_{\text{sub-ms ordering}} \parallel \underbrace{2 \text{ bits: variant}}_{\text{}} \parallel \underbrace{62 \text{ bits: random}}_{\text{}} \quad (55)$$

This ensures that the default `ORDER BY created_at` and `ORDER BY id` produce identical orderings, enabling future migration from timestamp-based to UUID-based ordering with no query changes.

## 20 Conclusions

This document has presented a complete mathematical specification of the National Addressing Reference System. The key formal results are:

1. The road angular-deviation bound (eq. (10)) guarantees that no road segment subtends an obtuse deflection angle, which is a necessary condition for unambiguous address assignment.
2. The district non-overlap (eq. (12)), adjacency (eq. (14)), and coverage (eq. (15)) conditions collectively ensure that the district partition is a topological valid tiling of the urban areas up to a 10 m tolerance.
3. The refresh-token rotation protocol (section 7.3) provides single-use guarantees for refresh tokens through row-level locking, preventing concurrent reuse without introducing distributed coordination.
4. The sliding-window rate limiter (eq. (30)) bounds the per-IP request rate, and the account-lockout mechanism (eq. (27)) provides a secondary defence against credential-stuffing attacks.
5. The road-direction algorithm (section 14) constructs a full topology graph handling T-junctions and X-junctions in  $O(n^2)$  time, then orients every segment via a depth-first search from the city-centre perimeter, with geographic fallback for isolated roads.
6. The house-entrance numbering scheme (section 15) is well-defined and consistent: arc-length projection combined with odd/even side assignment guarantees that no two entrances on the same side of a road share a number, and the seeding from already-assigned numbers prevents gaps in the sequence.

7. The circle ring computation (section 13) solves the spherical direct problem exactly, producing correct vertex positions at all latitudes without the cosine-scale approximation error of the naïve formula.
8. The even-odd ray-casting algorithm (section 16) provides  $O(n)$  point-in-polygon classification for both the municipality boundary and scattered area polygons, without any server round-trip.

Future work should consider migrating coordinate storage from JSON arrays to native PostGIS geometry columns, which would eliminate the coordinate-order swap in `SqlFragments.cs`, enable spatial indexing on feature geometries (currently only possible via `ST_GeomFromGeoJSON` at query time), and reduce storage overhead by approximately 40 %.

## Acknowledgements

The authors acknowledge the use of the following open-source projects: PostgreSQL, PostGIS, ASP.NET Core, Entity Framework Core, Vue 3, MapLibre GL JS, Geoman, Turf.js, DOMPurify, and vue-i18n.

## References

- [1] National Geospatial-Intelligence Agency (2014) Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems, 4th edn. NGA.STND.0036\_1.0.0\_WGS84, Springfield
- [2] Sinnott RW (1984) Virtues of the Haversine. *Sky and Telescope* 68(2):159
- [3] Provos N, Mazières D (1999) A Future-Adaptable Password Scheme. In: *Proc USENIX Annual Technical Conference, FREENIX Track*, Monterey, 81–91
- [4] Jones M, Bradley J, Sakimura N (2015) JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- [5] Obe R, Hsu L (2021) *PostGIS in Action*, 3rd edn. Manning, Shelter Island
- [6] Krawczyk H, Bellare M, Canetti R (1997) HMAC: Keyed-Hashing for Message Authentication. RFC 2104. Internet Engineering Task Force. <https://doi.org/10.17487/RFC2104>
- [7] MapLibre Contributors (2024) MapLibre GL JS. <https://maplibre.org>. Accessed 18 April 2026
- [8] Butler H, Daly M, Doyle A, Gillies S, Hagen S, Schaub T (2016) The GeoJSON Format. RFC 7946. Internet Engineering Task Force. <https://doi.org/10.17487/RFC7946>
- [9] OWASP Foundation (2023) Cross-Site Request Forgery (CSRF) Prevention Cheat Sheet. [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html). Accessed 18 April 2026

- [10] Leaky Bucket vs Token Bucket: Rate Limiting Algorithms (2022). Microsoft Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/patterns/rate-limiting-pattern>. Accessed 18 April 2026
- [11] Davis K, Peabody B, Leach P (2022) A New Format for Time-Ordered Universally Unique Identifier (UUID). RFC 9562. Internet Engineering Task Force. <https://doi.org/10.17487/RFC9562>